Agile:

First, it's crucial to understand that Agile is **not** a specific methodology with defined steps like the Waterfall model. Instead, it is a set of principles and values outlined in the **Agile Manifesto**.

The Agile Manifesto was created in 2001 by a group of leading software developers who were frustrated with the shortcomings of traditional, heavyweight models. They defined a better way of developing software.

Agile Manifesto

The Four Core Values of the Agile Manifesto:

This is the heart of Agile. It's a statement of priority, not a rejection of the items on the right.

1. **Individuals and Interactions** *over* processes and tools
    - The best architecture, requirements, and designs emerge from self-organizing teams. Communication is valued more than strict adherence to a process.
2. **Working Software** *over* comprehensive documentation
    - The primary measure of progress is delivering functional software to the customer. Documentation is still important, but it should be lean and just enough to support the product.
3. **Customer Collaboration** *over* contract negotiation
    - The customer is not an challenger to negotiate a fixed contract with. They are a collaborative partner throughout the project, providing continuous feedback.
4. **Responding to Change** *over* following a plan
    - Change is not only expected; it is welcomed. Agile processes fix change for the customer's competitive advantage. A project plan is a guide, not a rigid set of instructions.

The 12 Principles Behind the Agile Manifesto:

These principles put values into practice. Key themes include:

- Satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development.
- Deliver working software frequently (from a couple of weeks to a couple of months).
- Businesspeople and developers must work together daily.
- Build projects around motivated individuals. Trust them to get the job done.
- The most efficient and effective method of conveying information is face-to-face conversation.
- Working software is the primary measure of progress.
- Sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architecture, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

What is Agility:

Agility is the ability of an organization, team, or individual to be quick, flexible, and adaptable in response to changing conditions.

- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Making the customer as a team member
- Rapid, incremental delivery of software
- Organizing a team so that it is in control of the work performed
- Recognizes that plans are short-lived
- Develops software iteratively with heavy emphasis on construction activities
- Delivers multiple 'software increments'
- Adapts as changes occur

A Simple Analogy: The Sports Car vs. The Mining Truck

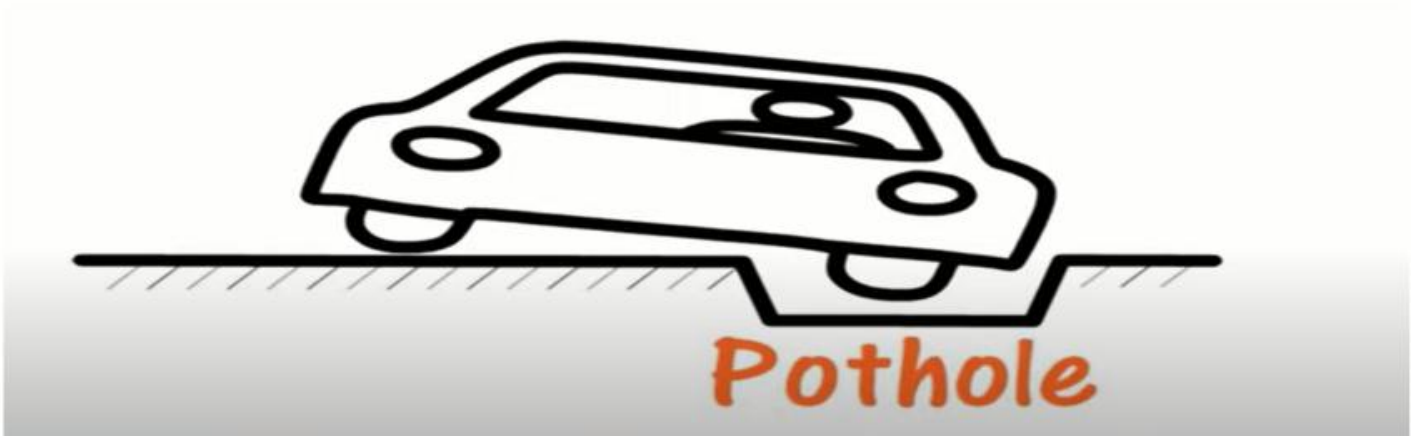
- A **Waterfall** process is like a massive **mining truck**. It's incredibly powerful and efficient for a single, well-defined task: moving a mountain of dirt from point A to point B on a planned road. But

if you need to change the destination (Point C) halfway through, it's incredibly expensive and difficult. It has to stop, turn around slowly, and a new road must be built.

- An **Agile** process is like a **high-performance sports car**. Its value isn't just its top speed. Its true value is its **handling, braking, and acceleration**—its ability to navigate a twisting, unpredictable road course *quickly and safely*. It can take a sharp turn (a change in requirements) at high speed without crashing because it was designed for adaptability.

Problem

You have been assigned a project to fill this pothole. How you will solve the problem of filling the pothole using Waterfall mode, Iterative model, Incremental model and agile model?



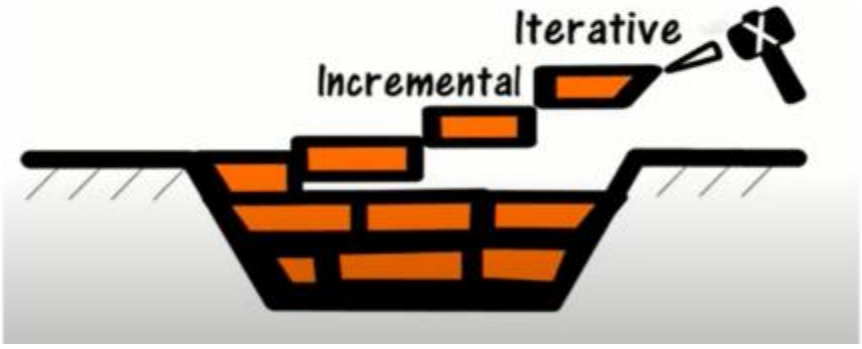
ITERATIVE APPROACH

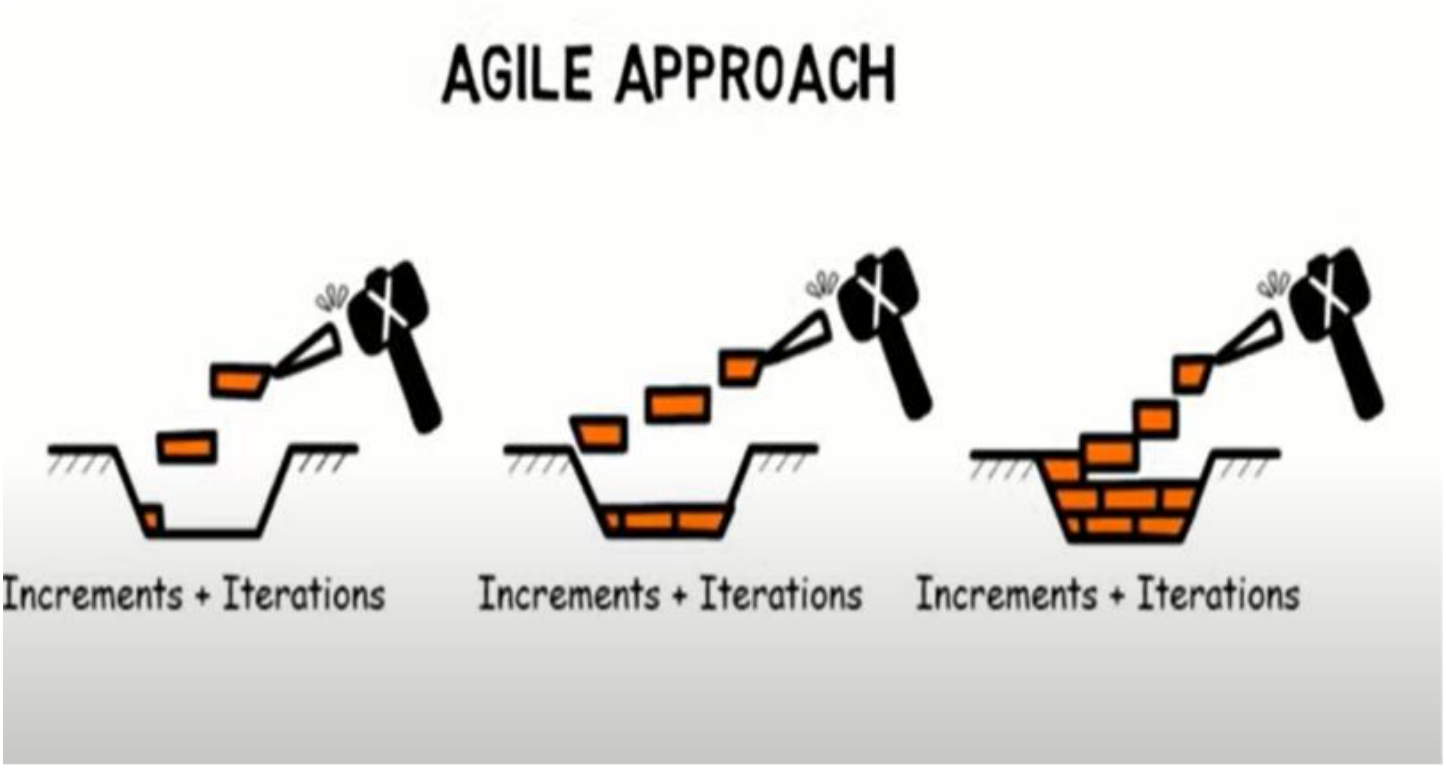
Iteration # 1 Iteration # 2 Iteration # 3

INCREMENTAL APPROACH

Increment # 1 Increment # 2 & 3 Increment # 4

Agile approach





1. Extreme Programming (XP):

XP is an Agile software development framework that aims to produce higher quality software, and higher quality of life for the development team, by fostering frequent releases in short development cycles, which improves productivity and introduces checkpoints where new customer requirements can be adopted.

It is the extreme approach of an iterative model:

- New versions may be built several times per day.
- Increments are delivered to customers every 2 weeks.
- All tests must be run for every build and the build is only accepted if tests run successfully.

Core XP Practices Shown:

1. **Pair Programming:** Two programmers work together at one workstation.
2. **Test-Driven Development (TDD):** Write tests before code to ensure quality and design.
3. **Continuous Integration (CI):** Merge and test code changes frequently to avoid integration hell.
4. **Small, Frequent Commits:** Break down work into tiny, manageable pieces.
5. **Customer Involvement:** A dedicated customer is available on-site as a team member who provides continuous feedback.
6. **Refactoring:** Continuously improve the code to make future changes easy.

XP Values:

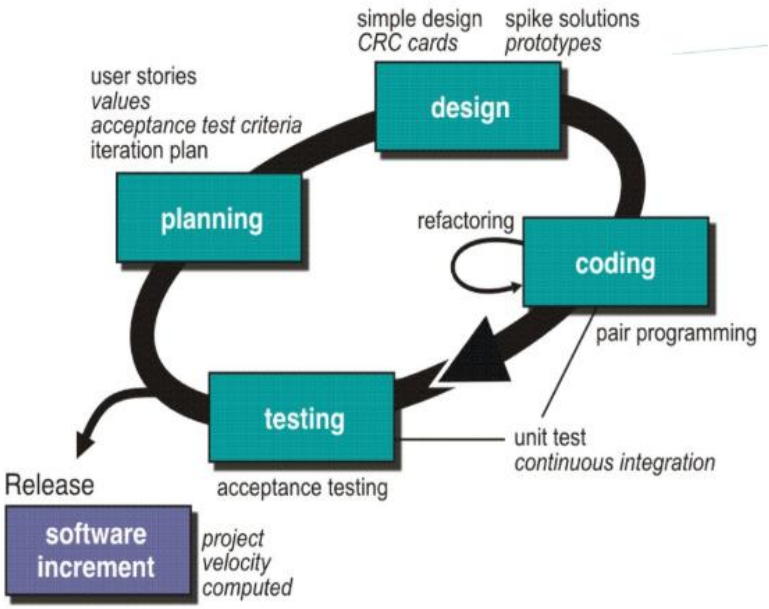
Communication: XP emphasizes constant, high-bandwidth communication between customers and developers

Simplicity: The goal is to build simple, elegant systems that solve today's problems brilliantly, not to over-engineer for a future that may never come.

Feedback: XP teams get feedback on multiple levels and at very short cycles

Courage: Developers need courage to refactor the code, throw away code that isn't working.

Respect: All team members respect each other's contributions, expertise, and time.



1. XP Planning (The "What")

- **User Stories:** Simple, customer-centric descriptions of features ("As a user, I want...").
- **Values:** The business value each story provides; used for prioritization.
- **Acceptance Test Criteria:** The customer-defined, testable conditions for a story to be "Done."
- **Iteration Plan:** The commitment for a short (1-2 week) cycle, listing stories to be completed.

2. XP Design (The "How")

- **Planning:** Micro-planning by developers on how to implement a specific story.
- **Designing:** Creating simple, effective designs for current needs.
    - ***CRC Cards (Class Responsibility Collaborator):*** A low-tech tool (index cards) for brainstorming object-oriented designs.
    - ***Simple Design:*** A principle of building only what is necessary right now.
- **Coding:** Writing production code.
- **Testing:** Verifying code works correctly (primarily through Unit Tests).

3. XP Coding (The "Tools")

- **Pair Programming:** Two developers at one workstation; one codes (Driver), one reviews (Navigator).

- **Unit Testing:** Writing automated tests for small code units, often *before* the code (Test-Driven Development).
- **Continuous Integration (CI):** Integrating and testing code changes multiple times a day to keep the software always in a working state.
- **Refactoring:** Continuously improving code structure without changing its behavior to maintain simplicity.
- **Spike Solutions/Prototypes:** Short, time-boxed experiments to research solutions or reduce technical risk (often thrown away after).

4. XP Testing (The Outcome)

- **Acceptance Testing:** Running automated tests against the Acceptance Test Criteria to confirm the story meets customer needs.
- **Release:** Deploying the working, tested, and accepted software to production.

XP is ideal when:

- Requirements are **vague or change frequently**.
- The project carries **high risk** (e.g., new technology, strict deadlines).
- You need to **build quality in** from the start.
- The team is **co-located** or has excellent remote collaboration tools.
- You have **strong customer involvement**.

Customer involvement

- Customer involvement is a key part of XP where the customer is part of the development team.
- **The role of the customer is:**
  - To help **develop stories** that define the requirements
  - To help **prioritize the features** to be implemented in each release
  - To help **develop acceptance tests** which assess whether or not the system meets its requirements.

Story card for document downloading

Downloading and printing an article
First, you select the article that you want from a displayed list. You then have to tell the system how you will pay for it - this can either be through a subscription, through a company account or by credit card. After this, you get a copyright form from the system to fill in and, when you have submitted this, the article you want is downloaded onto your computer. You then choose a printer and a copy of the article is printed. You tell the system if printing has been successful. If the article is a print-only article, you canÕt keep the PDF version so it is automatically deleted from your computer .

Requirements scenarios

- In XP, user requirements are expressed as scenarios or user stories.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

XP and change

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- It proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

Refactoring

- Refactoring is the process of code improvement where code is re-organised and rewritten to make it more efficient, easier to understand, etc.
- Refactoring is required because frequent releases mean that code is developed incrementally and therefore tends to become messy.
- Refactoring should not change the functionality of the system.
- Automated testing simplifies refactoring as you can see if the changed code still runs the tests successfully.

Testing in XP

- Test-first development.
- Incremental test development from scenarios.
- **User involvement** in test development and validation.
- **Automated test harnesses** are used to run all component tests each time that a new release is built.

Test case description

Test 4: Test credit card validity
Input: A string representing the credit card number and two integers representin the month and year when the card expires **Tests:** Check that all bytes in the string are digits Check that the month lies between 1 and 12 and the year is greater than or equal to the current year . Using the first 4 digits of the credit card number , check that the card issuer is valid by looking up the card issuer table. Check credit card validity by submitting the card number and expiry date information to the card issuer **Output:** OK or error message indicating that the card is invalid

Test-first development

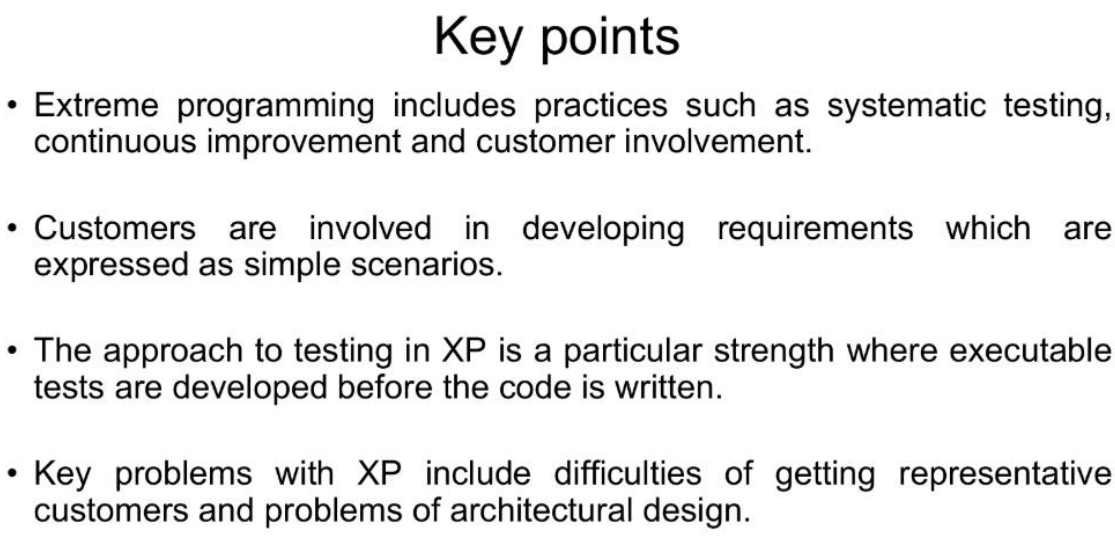
- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
- All previous and new tests are automatically run when new functionality is added. Thus checking that the new functionality has not introduced errors.

Pair programming

- In XP, programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this.
- Measurements suggest that development productivity with pair programming is similar to that of two people working independently.

Challenges of XP:

- Requires a **major cultural shift** and high discipline from the team.
- **Pair Programming** can be mentally exhausting and is often met with initial resistance.
- The **On-Site Customer** is difficult to arrange in many organizations.
- It can be perceived as too intense or "cult-like."
- Not all practices are suitable for every team or project; they're often adapted.
- **Customer involvement:** This is perhaps the most difficult problem. It may be difficult or impossible to find a customer who can represent all stakeholders and who can be taken off their normal work to become part of the XP team. For generic products, there is no 'customer' - the marketing team may not be typical of real customers.
- **Architectural design:** The incremental style of development can mean that inappropriate architectural decisions are made at an early stage of the process. Problems with these may not become clear until many features have been implemented and refactoring the architecture is very expensive.
- **Test complacency:** It is easy for a team to believe that because it has many tests, the system is properly tested. Because of the automated testing approach, there is a tendency to develop tests that are easy to automate rather than tests that are 'good' tests.



Classic Example Scenario for XP

Project: Develop a new **algorithmic stock trading engine** for a financial startup.

Let's analyze this scenario against our checklist:

- ✓ **Vague/Changing Requirements:** The traders have ideas for strategies, but they don't know which will work best. They will want to experiment, see results, and change the algorithms frequently. Requirements are highly dynamic.
- ✓✓✓ **High Risk:**
    - **Technical Risk:** Requires complex math, real-time data processing, and ultra-low latency—all technically challenging.
    - **Schedule Risk:** The startup needs to get to market before competitors.
    - **Requirements Risk:** The exact profitable algorithm is unknown.
- ✓ **Small, Co-located Team:** This would likely be a small, elite team of developers and quants.
- ✓✓✓ **Emphasis on Quality & Maintainability:** A single bug could lose millions of dollars. The codebase must be extremely reliable, simple, and easy to change safely. **This is a critical point.**
- ✓ **Dedicated, Accessible Customer:** The traders (the users) would be sitting *with* the development team, constantly refining strategies and providing feedback on the engine's performance.
- ✓ **Focus on Engineering Excellence:** This is non-negotiable. The technical practices of XP are perfectly fit.

Contrasting Example: When NOT to Use XP

Project: Develop the software for an **ATM (Automated Teller Machine)**.

Let's analyze this scenario:

- ✗ **Vague/Changing Requirements:** Requirements are extremely well-defined, fixed, and regulated upfront. (e.g., dispense cash, accept deposits, show balance). Changes are rare and go through a rigorous process.
- ✗ **High Risk (of a different kind):** The risk is mitigated by extensive, upfront planning and documentation, not by emerging code. The risk is in *not* following the plan.
- ✓ **Emphasis on Quality:** Yes, quality is critical. However, this quality is achieved through rigorous upfront specification and testing phases (V-Model), not emergent design.

- ✗ **Dedicated, Accessible Customer:** The "customer" is a large bank's committee. They will not be sitting with developers. They will provide a 500-page specification document and sign off on stages.

Exam Answer Template

If an exam question asks, **"Which model is most suitable?"** and the scenario matches the XP checklist, structure your answer like this:

"Extreme Programming (XP) would be the most suitable model for this project. This is because:

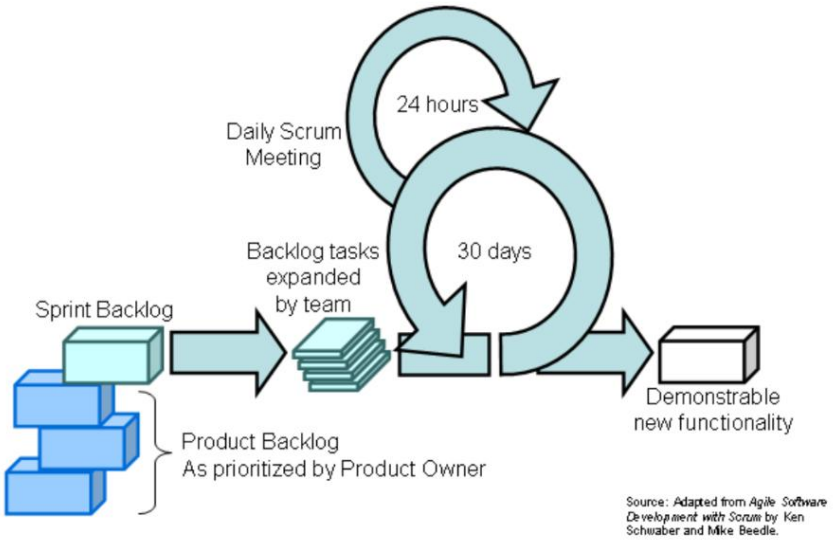
1. The requirements are **[vague/expected to change rapidly],** as stated in the scenario.
2. There is significant **[technical/schedule]** risk involved.
3. The project demands very **high code quality** and the ability to **change the software safely**, which is addressed by XP's core technical practices like **Test-Driven Development, Refactoring, and Continuous Integration (CI).**
4. The presence of a dedicated, **on-site customer** will enable the continuous feedback and collaboration that XP requires.
   Therefore, XP's iterative, feedback-driven, and quality-focused approach is the best fit for this context."

Spiral vs. XP

Aspect	**Spiral Model**	**Extreme Programming (XP)**
Model Type	**Predictive** (Plan-driven)	**Agile** (Change-driven)
Core Philosophy	**Risk-Driven.** Develop through repeated cycles (spirals) of planning, risk analysis, and prototyping.	**Customer & Code-Centric.** Produce high-quality software that adapts to changing requirements through technical excellence and constant feedback.
Primary Focus	**Identifying and mitigating risks** early in the project.	**Satisfying the customer** through rapid delivery of valuable software and **technical excellence**.
Approach to Change	Change is difficult and expensive. Requires formal re-evaluation and re-planning.	**Embraces change.** Requirements are expected to evolve, even late in development.
Project Size	Best for **large, complex, high-risk** projects where **requirements are vague** (e.g., flight control systems).	Best for **small to medium-sized** teams working on projects with **vague or changing requirements**.
Key Practices	1. Plan 2. Risk Analysis 3. Engineering (Develop & Test) 4. Customer Evaluation	1. **Test-Driven Development (TDD)** 2. **Pair Programming** 3. **Continuous Integration** 4. **Small Releases** 5. **On-Site Customer**
Documentation	**Extensive.** Heavy documentation is required for risk analysis and planning.	**Minimal.** Code is the primary artifact. Focus is on working software over comprehensive documentation.
Customer Involvement	Periodic, mainly at the end of each spiral for evaluation.	**Constant.** An on-site customer is part of the team, providing continuous feedback.

2. Scrum:

Scrum employs an **iterative and incremental approach** to **optimize predictability and control risk**.



1. Roles & Key Responsibilities

Role	Key Responsibility	Key Traits
Product Owner	**Maximizes product value.** Manages and prioritizes the Product Backlog. Decides "What" to build and release date.	Voice of the customer. Single source of truth for requirements.
Scrum Master	**Facilitator: Removes impediments/obstruction.** Ensures the team follows Scrum processes and protects them from distractions.	Servant-leader. Coach and facilitator for the team and organization.
Development Team	**Delivers a "Done" product increment each Sprint.** Self-organizes to determine "How" to do the work.	Cross-functional (has all necessary skills). Self-organizing. 3-9 members.

2. Practices (Events)

All events are **time-boxed**.

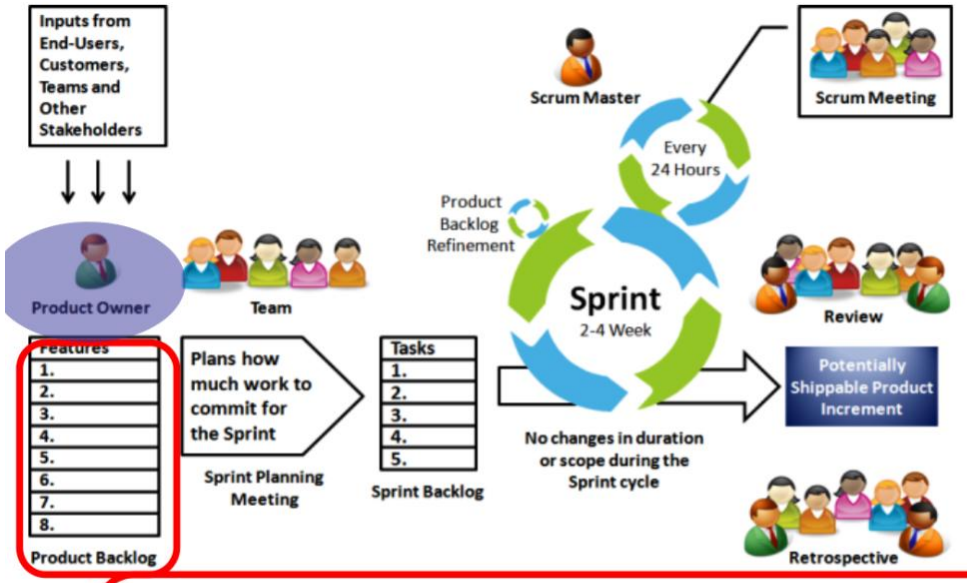
Event	Purpose	Key Outcome
Sprint Planning	Plan the work for the upcoming Sprint. Product Owner describes highest priority features to the Team. Team decides what they can commit to deliver in the Sprint.	A Sprint Goal and a Sprint Backlog.
The Sprint	The **heart of Scrum**. A time-box (2-4 week) to create a **done**, usable increment.	A potentially shippable product increment. **No scope changes during.**
Daily Scrum	The **Team** and the **Scrum Master** only. 15-minute time-box, to inspect progress and plan the next 24 hours.	Improved team coordination and a plan for the day.
Sprint Review	Time boxed to one hour of prep and four hours of meeting. Team demonstrates product to product owner who inspect the increment and gather feedback from stakeholders.	Adapted Product Backlog based on feedback.
Sprint Retrospective	Team, Scrum Master, and (optionally) Product Owner review the last sprint. Inspect the team's process and create a plan for improvements in the next Sprint.	Concrete process improvements for the next Sprint.

3. Artifacts

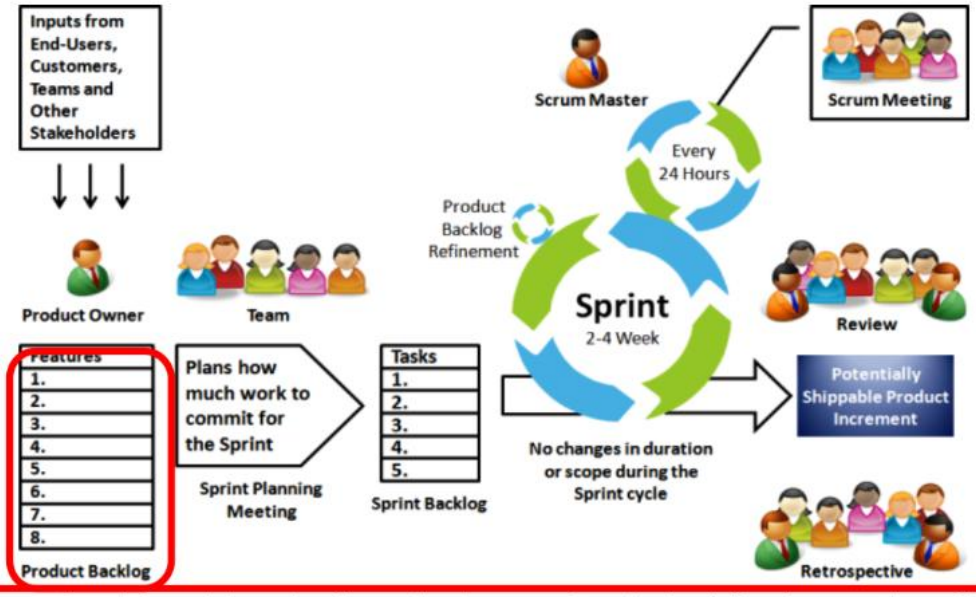
Designed to maximize **transparency**.

Artifact	Purpose	Key Detail
Product Backlog	An **ordered (prioritized) list** of all desired work on the product.	Dynamic and never complete. Owned by the Product Owner. Constantly changes based on feedback and stakeholder desires.

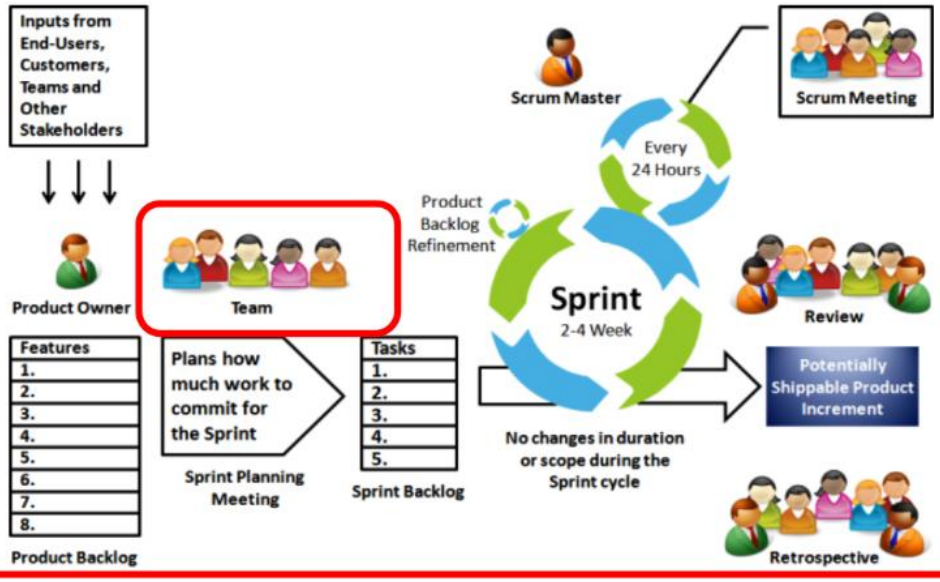
Artifact	Purpose	Key Detail
Sprint Backlog	The set of Product Backlog items selected for the Sprint + the team's plan to achieve the Sprint Goal.	Owned and managed by the Development Team. Doesn't change after planned.
Product Increment	The sum of all completed Product Backlog items from the current Sprint **plus** all previous Sprints.	Must be "Done" (meet the team's shared **Definition of Done**).
Burndown Chart	A graph showing the remaining work in the Sprint Backlog.	A tool for the team to inspect their progress and self-manage.



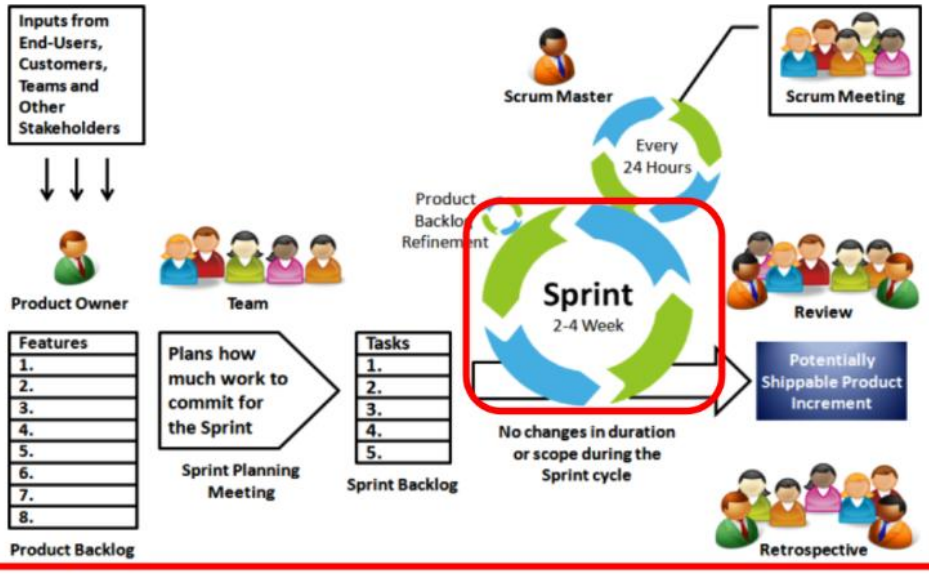
- Product Owner decides what should be produced so as to achieve success
- Gets inputs from users, teams, managers, stakeholders, executives, industry experts etc.
- Produces the product backlog which contains the features of product to be produced in order of priority



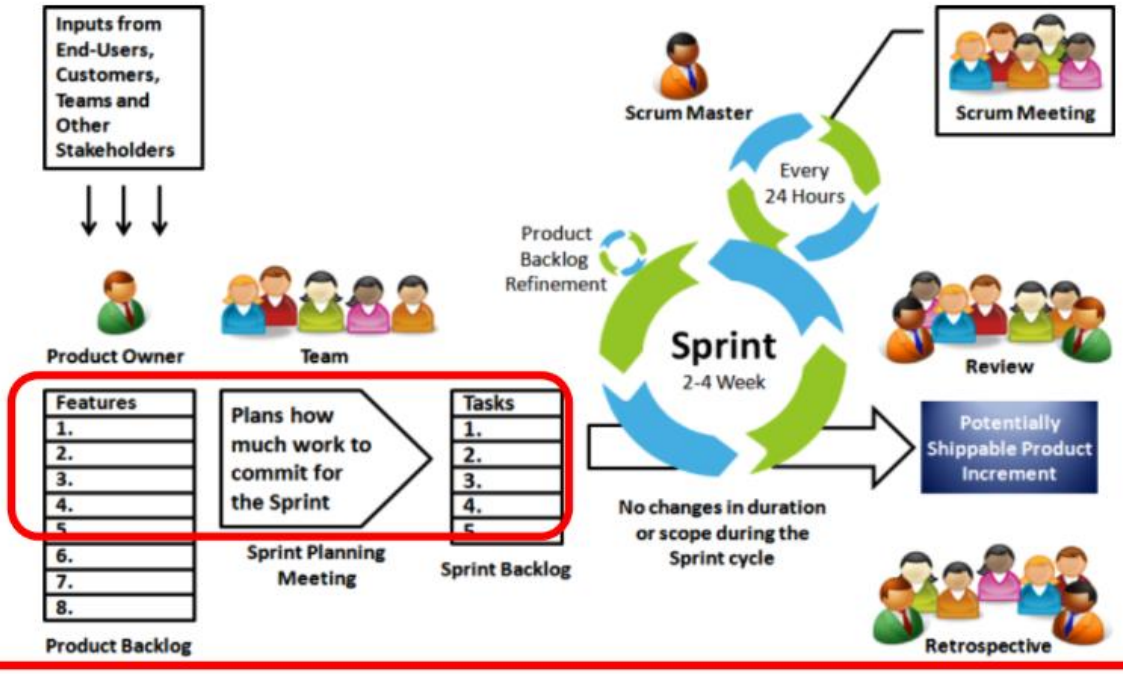
- The **product backlog** is the single master list of features, functionality etc ordered based on business value and risk.
- The product backlog is **constantly being revised** by the product owner, to maximize the business success of the team's efforts



- Consists of **7** to **9** people
- Team has to be cross functional- containing members from different verticals required for developing the product.
- Team is self organized and self managing – makes a commitment and manages its responsibilities.



- Sprints is referred to the fixed period of time the team commits to work in course of developing the product
- The length of sprint is decided by the team and product owner
- Working as sustainable pace is important to avoid burn out



- Team selects what it will to deliver by the end of sprint.
- Every member of team take part

Difference between two agile processes

1. Scrum teams have sprint which are 2-4 weeks long whereas XP teams typically work in iteration no longer than 1-2 weeks long.

2. Scrum doesn't allow changes to the chosen sprint backlog once planned whereas XP teams XP teams are much more amenable to change within their iterations, but change can only be made if the team hasn't started working on a feature.

3. In Scrum product owner prioritizes the product backlog but the Scrum team can choose a lower priority item for a sprint to work on before the high priority work whereas XP teams must always work in priority order as Features to be developed are prioritized by the customer

The Biggest difference are

XP mandates a set of engineering practices where Scrum does not. At the same time Scrum mandates planning ceremonies and artifacts, where XP does not.

Factors	Waterfall	V-shaped	Spiral	Evolutionary Prototyping	Incremental & Iterative	Agile Methodologies
Unclear User Requirements	Poor	Poor	Excellent	Good	Good	Excellent
Unfamiliar Technology	Poor	Poor	Excellent	Excellent	Good	Poor
Complex System	Good	Good	Excellent	Excellent	Good	Poor
Reliable System	Good	Good	Excellent	Poor	Good	Good
Short Time Schedule	Poor	Poor	Excellent	Good	Excellent	Excellent
Strong Project Management	Excellent	Excellent	Excellent	Excellent	Excellent	Excellent
Cost Limitation	Poor	Poor	Poor	Poor	Excellent	Excellent
Visibility of Stakeholders	Good	Good	Excellent	Excellent	Good	Excellent
Skills Limitation	Good	Good	Poor	Poor	Good	Poor
Documentations	Excellent	Excellent	Good	Good	Excellent	Poor
Component Reusability	Excellent	Excellent	Poor	Poor	Excellent	Poor